Embedded Respiratory Breath Classification Using Lightweight Machine Learning on ESP32-S3

Upani Gunathunga
Biomedical Engineering Undergraduate
Department of Electronic and Telecommunication Engineering
University of Moratuwa, Sri Lanka

2026

Abstract

Respiratory sound analysis is becoming an important tool in modern healthcare for monitoring breathing behaviour and detecting abnormal respiratory patterns. In this project, a complete embedded machine learning pipeline was developed to classify breathing patterns in real time using an ESP32-S3 microcontroller together with a digital MEMS microphone.

The system records respiratory audio signals and extracts a set of lightweight statistical features from the captured data. These features are then used by a multinomial logistic regression model that performs the classification directly on the microcontroller. The design focuses on keeping the computations simple and efficient so that the entire system can run within the limited memory and processing capabilities of an embedded device.

This report describes the full development process of the system, starting from the selection of hardware components and preparation of the dataset, through the design of the signal processing pipeline and the development of the machine learning model, and finally the deployment of the model on the embedded platform. It also discusses the key design decisions made during the project and the trade-offs that arise when adapting machine learning techniques to resource-constrained embedded systems.

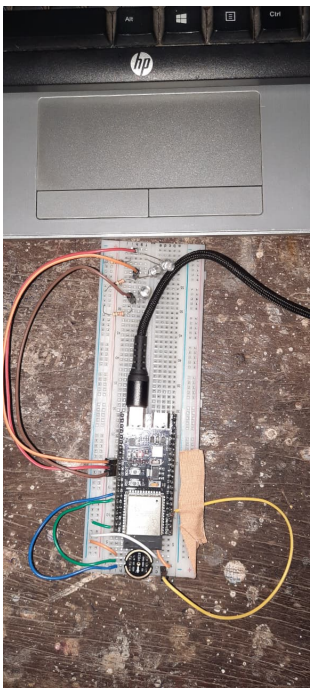

Figure 1: The hardware circuit diagram

1 Introduction

Respiratory signals contain valuable information about the state of a person's breathing activity. In clinical environments, respiratory sound analysis is often used to detect conditions such as wheezing, crackles, or abnormal breathing rhythms. However, most existing systems rely on computationally intensive signal processing or cloud-based machine learning.

The motivation behind this project was to explore whether a low-cost embedded system could perform meaningful respiratory signal interpretation locally, without relying on external computing resources.

The goal was therefore to design a system capable of:

- capturing respiratory sounds in real time
- extracting meaningful signal features
- performing machine learning inference directly on the microcontroller
- providing immediate feedback through a simple interface

A practical embedded pipeline was implemented using the ESP32-S3 microcontroller and an INMP441 digital MEMS microphone.

The final system classifies breaths into three categories:

- **short breaths - Red LED**
- **normal breaths - Blue LED**
- **long breaths - Green LED**

These classifications are indicated using LED outputs.

2 System Overview

The system architecture consists of several sequential processing stages.

Respiratory Sound → Microphone → Feature Extraction → ML Classifier → LED Output

Audio signals are captured by the INMP441 microphone using the ESP32's I2S peripheral. Breath segments are detected using an energy-based state machine. Once a breath segment has been captured, statistical features are computed and passed into a trained logistic regression classifier.

The classifier determines the most likely breath class and activates the corresponding LED indicator.

The entire pipeline runs locally on the ESP32-S3 without requiring external computation.

2.1 Signal Processing Pipeline

Respiratory signals are captured using a digital sampling process before feature extraction and classification.

The system records audio at a sampling rate of:

16 kHz

This sampling rate was selected based on several signal processing considerations.

Respiratory acoustic energy typically occupies a relatively low frequency range compared to speech signals. Most informative respiratory sound components exist well below 8 kHz. Therefore, a 16 kHz sampling rate satisfies the Nyquist sampling criterion while still preserving the relevant acoustic characteristics of breathing sounds.

Choosing this sampling rate also provides several practical advantages for embedded systems:

- reduced memory consumption for audio buffers
- lower computational load during feature extraction
- sufficient temporal resolution for breath detection

These factors make 16 kHz a suitable balance between signal fidelity and embedded system efficiency.

3 Hardware Design Decisions

Selecting the appropriate hardware platform was an important design step because embed-

ded machine learning workloads impose constraints on memory, processing power, and peripheral capabilities.

Initially, development began on a standard NodeMCU ESP32 board. However, during early experiments it became clear that the available memory was insufficient for reliable audio buffering and feature extraction.

Respiratory sounds require continuous sampling and buffering in order to capture complete breathing cycles. These buffers can quickly consume a large portion of available RAM.

For this reason the ESP32-S3 N1R8 board was selected as the final platform. The ESP32-S3 provides several advantages:

- 240 MHz dual-core processor
- significantly larger SRAM
- optional PSRAM support
- improved I2S peripheral interface

These features make the ESP32-S3 particularly suitable for embedded signal processing and lightweight machine learning workloads.

4 Microphone Selection

The INMP441 digital MEMS microphone was chosen primarily because it provides a native I2S digital interface.

Many low-cost microphones produce analog signals that must be digitized using an ADC. However, ADC sampling introduces additional complexity such as amplification circuits, filtering stages, and noise susceptibility.

The INMP441 eliminates these requirements by providing a digital audio stream directly to the microcontroller.

This simplifies the hardware design and improves signal stability.

5 Dataset Selection

The machine learning model used in this project was trained using the publicly available Respiratory Sound Database.

This dataset contains over 900 respiratory recordings collected from multiple patients in clinical settings.

Each recording includes annotations identifying respiratory cycles and indicating the presence of abnormal sounds such as crackles or wheezes.

For this project only clean breathing cycles were used. **Cycles containing wheezes or crackles were removed in order to ensure that the model learned differences in breathing duration rather than pathological noise patterns.**

This decision simplified the classification task and made the model more suitable for the intended proof-of-concept system.

6 Feature Engineering [1]

Feature engineering was one of the most critical parts of the design process.

In many audio classification tasks, Mel-Frequency Cepstral Coefficients (MFCCs) are commonly used as input features. MFCCs capture detailed spectral characteristics of audio signals and are widely used in speech recognition systems.

During early experimentation MFCC features were initially considered for this project as well.

However, implementing MFCC extraction on a microcontroller presents several challenges:

- multiple FFT operations are required
- significant memory is needed for frame buffers
- computational latency increases
- model input dimensionality becomes large

These factors significantly increase computational complexity and memory usage.

Since the goal of this project was to perform real-time inference directly on the ESP32, a simpler feature representation was required.

Therefore a set of lightweight statistical features was designed instead.

The final feature set consists of four features:

- **Root Mean Square (RMS) energy**
- **Zero Crossing Rate (ZCR)**
- **Signal variance**
- **Breath duration**

These features are computationally inexpensive and can be calculated using basic arithmetic operations.

Despite their simplicity, these features still capture meaningful characteristics of respiratory sounds such as intensity, oscillation behaviour, and temporal duration.

6.1 Mathematical Definition of Selected Features

To further clarify the design of the lightweight feature extraction pipeline, the mathematical definitions of the selected signal features are summarized below.

Root Mean Square (RMS) Energy

RMS energy measures the overall intensity of the signal and is widely used in audio signal processing to estimate signal power. It is defined as:

$$RMS = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2} \tag{1}$$

where x_i represents the audio sample value and N is the number of samples in the captured breath segment.

Respiratory airflow generates acoustic energy within the microphone, therefore the RMS value provides a useful estimate of breathing intensity.

Zero Crossing Rate (ZCR)

The Zero Crossing Rate measures how frequently the signal waveform crosses the zero amplitude axis. It is calculated as:

$$ZCR = \frac{\text{Number of sign changes}}{N} \tag{2}$$

Respiratory sounds typically exhibit characteristic oscillation patterns depending on airflow turbulence and microphone placement. The ZCR feature captures these oscillatory characteristics with minimal computational overhead.

Signal Variance

Signal variance measures the spread of amplitude values within the breathing segment. It captures how much the signal amplitude fluctuates during a breath cycle. Respiratory sounds often contain varying intensity patterns due to inhalation and exhalation airflow dynamics, and variance provides a compact statistical representation of this variability.

Breath Duration

Breath duration represents the total time length of the detected breathing event, measured in seconds.

This feature plays a critical role in distinguishing between the three breathing classes used in this system. Short breaths, normal breaths, and long breaths primarily differ in their temporal duration, making this feature one of the most discriminative inputs for the classifier.

7 Machine Learning Model Design

The classification model used in this system is a **multinomial logistic regression classifier.**

Several machine learning models were considered during development, including decision trees and small neural networks. However, these models either increased memory usage or required more complex inference operations.

Logistic regression was selected because it offers several advantages for embedded systems:

- small memory footprint
- deterministic inference
- simple linear computations
- easy parameter export to C code

The trained model produces a weight matrix and bias vector which can be embedded directly into the firmware.

In order to deploy the trained model on the microcontroller, the model parameters were exported from the Python training environment and converted into static C arrays.

The exported parameters include:

- the feature normalization mean vector
- the feature normalization standard deviation vector
- the logistic regression weight matrix
- the bias vector for each class

During inference, the ESP32 firmware uses these parameters to normalize the extracted features and compute the linear classification scores. Because logistic regression relies only on simple arithmetic operations, the entire inference procedure can be implemented using lightweight numerical computations without requiring complex neural network frameworks. During inference the microcontroller simply computes linear scores for each class and selects the highest value.

This design greatly simplifies deployment and ensures deterministic execution on the microcontroller.

This approach allows machine learning classification to be performed with minimal computational overhead.

8 Model Performance Evaluation

The trained logistic regression model was evaluated using a held-out test dataset to assess its classification performance.

Test Accuracy

The model achieved an overall test accuracy of approximately **97%**, indicating that the selected feature set and model architecture are effective for distinguishing between the defined breathing classes.

Confusion Matrix Analysis

The confusion matrix results are summarized below:

- **Long breaths** – 339 correctly classified samples
- **Normal breaths** – 289 correctly classified samples
- **Short breaths** – 72 correctly classified samples

Both precision and recall remain high across the three classes, demonstrating stable classification behaviour.

A slightly lower precision was observed for the short breath category. This is likely due to the fact that short breath samples were comparatively underrepresented in the training dataset. Class imbalance can influence decision boundaries in linear classifiers such as logistic regression.

Despite this minor imbalance, the overall classification performance remains strong and sufficiently reliable for the proof-of-concept embedded implementation.

9 Embedded Implementation

Once the model was trained in Python, its parameters were exported and embedded into the ESP32 firmware. **The inference pipeline follows several steps:**

1. detect a breath segment
2. normalize the signal
3. compute statistical features
4. normalize features using training statistics
5. compute classifier scores
6. select the predicted class

This pipeline allows classification to occur immediately after a breath is detected.

The result is displayed using LED indicators connected to the ESP32.

10 Challenges and Engineering Considerations

10.1 Automatic Noise Calibration

Environmental noise can significantly affect the reliability of breath detection in real-world environments. Background sounds may occasionally produce signal energy levels similar to

weak breathing signals, which can lead to false detections.

To address this issue, the system performs an automatic calibration procedure during startup.

The calibration process follows three steps:

1. The system records a short segment of ambient background audio immediately after power-up.
2. RMS statistics are computed from the captured background signal.
3. A dynamic silence threshold is then calculated based on the observed noise level.

This adaptive threshold allows the system to adjust to different environments such as quiet rooms or slightly noisy indoor spaces.

By calibrating the detection threshold automatically, the system improves robustness and reduces the likelihood of false breath detections caused by environmental noise.

10.2 Memory and Buffer Management

Memory management was one of the most significant challenges. Breath signals had to be buffered while still leaving enough memory available for the runtime system.

Efficient memory planning was a critical part of the embedded system design.

Breath signals are temporarily stored using a circular audio buffer implemented in RAM. The buffer size was configured to store up to:

40,000 audio samples

At the selected sampling rate of **16 kHz**, this buffer size supports breath durations of approximately **2.5 seconds** while maintaining a safe margin for processing overhead.

This design ensures that complete breathing cycles can be captured without risking buffer overflow while still preserving sufficient memory for the runtime system, feature extraction computations, and machine learning inference.

The availability of additional SRAM and optional PSRAM on the ESP32-S3 was therefore an important factor in selecting this microcontroller for the project.

Noise calibration was another important consideration. Environmental noise can easily trigger false breath detections. To address this, the system performs an automatic calibration step during startup to determine a suitable silence threshold.

Additionally, real-time operation required careful tuning of buffer sizes, detection thresholds, and state machine logic.

11 System Limitations

Although the system successfully demonstrates real-time embedded breath classification, several design limitations should be acknowledged.

- The current model primarily detects differences in **breath duration patterns**. It does not attempt to classify pathological respiratory sounds such as wheezes, crackles, or other clinically relevant anomalies.
- The system assumes that the **microphone is positioned relatively close to the user's mouth**. Greater distances may reduce signal quality and affect feature extraction reliability.
- **Environmental noise** can influence breath detection accuracy. Although an automatic noise calibration step is performed during startup, sudden environmental disturbances may still trigger false detections.
- The dataset used for training contains fewer short-breath examples compared to other classes, which may introduce minor class imbalance during classification.

These limitations highlight potential areas for future improvement, such as incorporating more diverse datasets, implementing noise-robust detection algorithms, and exploring additional signal features.

12 Conclusion

This project demonstrates how embedded machine learning can be applied to interpret physiological signals in real time using low-cost hardware.

By carefully selecting lightweight features and a computationally efficient model, it was possible to implement a complete machine learning pipeline directly on a microcontroller.

The final system successfully captures respiratory audio, detects breathing events, and classifies breathing patterns in real time.

This work highlights the potential for embedded machine learning systems to support portable health monitoring devices in the future.

References

[1] J. Fernandes, B. Rocha, R. P. Paiva, and T. Cruz, "Using Low Cost Embedded Systems for Respiratory Sounds Auscultation," in *Proceedings of the International Symposium on Wireless Communication Systems (ISWCS)*, 2018. doi: 10.1109/ISWCS.2018.8491232.